CORTEX Protocol — Developer Handoff & Implementation Guide

Version: 1.0 **Date:** March 20, 2026 **Status:** Pre-Launch (Frontend Complete, Backend Partial, Smart Contracts Not Deployed)

Table of Contents

1. Project Overview
2. Current State Summary
3. Architecture
4. Tech Stack
5. Critical Issues (Why Mock Data Still Shows)
6. Supabase Database Schema
7. Smart Contracts (Must Deploy)
8. AI Agent System
9. Cron Jobs (All Placeholder)
10. Frontend → API Mapping
11. Environment Variables
12. Implementation Roadmap
13. Deployment

---

1. Project Overview

CORTEX is a decentralized, AI-managed investment vault protocol on **Base L2** (Chain ID 8453). Users deposit ETH or USDC, receive ERC-4626 cVault shares, and an autonomous AI agent manages a diversified portfolio across Base DeFi protocols (Aave, Compound, Aerodrome, Uniswap).

Live URL: https://www.cortexprotocol.net **GitHub:** https://github.com/CortexProtocolBase/Cortex-Protocol **Vercel Project:** https://vercel.com/bryces-projects-72528c60/cortex **Supabase:** https://qyqjydtmhumtpxfwlfbx.supabase.co

Core Concepts

- **cVault Shares:** ERC-4626 receipt token. Deposit ETH/USDC → receive cVLT proportional to vault share. Value increases as AI generates returns.
- **Three Tiers:** Core (70% — Aave/Compound yields), Mid-Risk (20% — AERO/DEGEN LPs), Degen (10% — new launches)
- **AI Agent:** Runs every 10 minutes. Analyzes on-chain data, sentiment, and liquidity. Proposes trades validated by smart contract guardrails.
- **Governance:** CORTEX token holders vote on risk parameters, fees, whitelisted strategies.
- **Fee Structure:** 2% management, 20% performance (above high-water mark), 0.5% withdrawal, 0% deposit. 50% fees → stakers, 50% → treasury.

---

2. Current State Summary

Component	Status	Notes
Frontend (Next.js)	✅ Complete	All 8 pages built with full UI
Frontend → API wiring	✅ Complete	All pages fetch from API routes
API Routes	⚠️ Partial	Query Supabase but fall back to mock data on any error
Supabase Tables	⚠️ Seeded Only	Tables exist with sample data, but schema may have mismatches
Supabase RLS Policies	❌ Not Configured	Public anon key may lack SELECT permissions
Smart Contracts	❌ Not Deployed	All contract addresses are `0x000...000`

AI Agent	❌ Not Built	No autonomous agent exists yet
Cron Jobs	❌ Placeholder	All 5 cron routes are TODO skeletons
Deposit/Withdraw	❌ Not Functional	UI exists but no smart contract to interact with
Token Gate	❌ Broken	Queries balance on zero address → always returns false
On-chain Governance	❌ Not Deployed	DB-only, no on-chain voting

Why the website shows mock data

Every API route follows this pattern:

```
try {
  const { data, error } = await supabase.from("table").select("*");
  if (error || !data) throw error;
  // ... map data to response
  return NextResponse.json({ data: mappedData });
} catch {
  // SILENT FALLBACK — no error logging
  return NextResponse.json({ data: mockData, cached: true });
}
```

The Supabase queries are failing silently because:

1. **RLS policies** may block the anon key from reading tables
2. **Column name mismatches** between code and actual DB schema
3. **Missing data** in tables
4. **No error logging** makes it impossible to debug

---

3. Architecture

```
┌──────────────────────────────────────────────────────────┐
│                      FRONTEND (Next.js)                  │
│  Landing | Dashboard | Vault | Trades | AI | Gov | Stake  │
│  Strategy                                                │
└──────────────────────────────────────────────────────────┘
                        │ fetch("/api/v1/...")
                        ▼
┌──────────────────────────────────────────────────────────┐
│                    API ROUTES (Next.js)                   │
│  /vault/stats  /trades  /ai/insights  /governance/proposals  │
│  /vault/user/[addr]  /portfolio  /ai/reasoning-feed      │
│  /staking/info/[addr]  /performance                      │
└──────────────────────────────────────────────────────────┘
                        │ supabase.from("table").select()
                        ▼
┌──────────────────────────────────────────────────────────┐
│                  SUPABASE (PostgreSQL)                   │
│  vault_snapshots | trades | ai_reasoning_logs            │
│  user_positions | deposits | withdrawals                 │
│  governance_proposals | staking_positions                │
└──────────────────────────────────────────────────────────┘
                        │ (data written by...)
                        ▼
┌──────────────────────────────────────────────────────────┐
│              DATA SOURCES (NOT YET BUILT)                 │
│                                                          │
```

```
│  🤖 AI Agent          — reads chain data, writes trades      │
│  📊 Cron: snapshot    — reads vault contract, writes stats   │
│  💰 Cron: fees        — calculates/distributes fees          │
│  📈 Cron: prices      — fetches token prices from DEX        │
│  🗳 Cron: governance  — syncs on-chain votes                 │
│  📉 Cron: analytics   — aggregates performance metrics       │
└──────────────────────────────────────────────────────────────┘
                        │
                        │ (reads from / writes to...)
┌──────────────────────▼───────────────────────────────────────┐
│             BASE L2 SMART CONTRACTS (NOT DEPLOYED)           │
│                                                              │
│  CortexVault.sol  — ERC-4626 vault (deposit/withdraw)        │
│  CortexToken.sol  — ERC-20 governance token                  │
│  Staking.sol      — Lock CORTEX, earn fee share              │
│  Governor.sol     — On-chain proposal voting                 │
│  Treasury.sol     — Fee collection + distribution            │
└──────────────────────────────────────────────────────────────┘
```

---

4. Tech Stack

Layer	Technology
Frontend	Next.js 16 (App Router), TypeScript, TailwindCSS
Charts	Recharts
Wallet	Wagmi v3, Viem, Phantom + Coinbase connectors
Database	Supabase (PostgreSQL + Realtime)
Hosting	Vercel (auto-deploy from GitHub)
Chain	Base L2 (Chain ID 8453)
Smart Contracts	Solidity (NOT YET WRITTEN)
AI Agent	NOT YET BUILT — needs Python or Node.js service

---

5. Critical Issues

Issue 1: Smart Contracts Not Deployed

File: `src/lib/constants.ts`

All contract addresses are zero addresses:

```
export const CONTRACTS = {
  CORTEX_TOKEN: "0x0000000000000000000000000000000000000000",
  CVAULT: "0x0000000000000000000000000000000000000000",
  STAKING: "0x0000000000000000000000000000000000000000",
  GOVERNANCE: "0x0000000000000000000000000000000000000000",
  TREASURY: "0x0000000000000000000000000000000000000000",
};
```

Impact:

- Deposits/withdrawals don't work (no vault contract to send transactions to)

- Token gate check (`src/lib/token-gate.ts`) always returns `false` because it queries `balanceOf()` on the zero address
- AI and Reasoning Feed API endpoints return 403 for all users (token gate fails)
- Staking is non-functional
- Governance voting is non-functional

Fix: Deploy all 5 contracts to Base, update addresses in `constants.ts` .

Issue 2: Token Gate Blocks AI Endpoints

Files: `src/app/api/v1/ai/insights/route.ts` , `src/app/api/v1/ai/reasoning-feed/route.ts`

Both AI endpoints require `x-wallet-address` header and call `getWalletFromHeaders()` . The frontend pages (`src/app/ai/page.tsx`) send this header from `useAccount()` . However:

- If no wallet is connected → shows "Connect wallet" message (correct behavior)
- If wallet is connected but token gate fails → returns 403 error

Currently the AI routes only check for header presence (not token balance), so they work IF the wallet is connected. But `hasTokenAccess()` is available for future gating.

Current state: AI endpoints work with wallet connected. The `getWalletFromHeaders` just validates the header format, doesn't check balance.

Issue 3: Supabase RLS (Row Level Security)

File: `src/lib/supabase.ts`

The public client uses the anon key. If RLS is enabled on tables (Supabase default), SELECT queries will return empty results unless policies explicitly allow reads.

Fix: In Supabase Dashboard → Authentication → Policies:

- Add `SELECT` policy for all tables: `true` (allow public reads)
- Or disable RLS on read-only tables

Issue 4: Possible Column Name Mismatches

The API routes expect specific column names. If the Supabase tables were created with different names, all queries fail silently.

Required columns per table — see Section 6.

Issue 5: No Error Logging

All API catch blocks silently return mock data with no `console.error()` . Add logging:

```
} catch (err) {
  console.error("[vault/stats] Supabase error:", err);
  return NextResponse.json({ data: mockVaultStats, cached: true });
}
```

Issue 6: Deposits/Withdrawals Not Wired

File: `src/app/vault/page.tsx`

The "Deposit to Vault" and "Redeem cVault Shares" buttons exist but have no `onClick` handler that sends a blockchain transaction. This requires:

1. A deployed ERC-4626 vault contract
2. Wagmi `useWriteContract` hooks for `deposit()` and `redeem()`
3. Token approval flow for USDC deposits

4. Transaction confirmation + Supabase record creation

---

6. Supabase Database Schema

Below are the **exact table schemas** the API routes expect. Create these tables with these exact column names and types.

vault_snapshots

```sql
CREATE TABLE vault_snapshots (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  timestamp TIMESTAMPTZ DEFAULT now() NOT NULL,
  total_assets NUMERIC NOT NULL,          -- TVL in USD
  share_price NUMERIC NOT NULL,           -- Price per cVault share
  total_shares NUMERIC NOT NULL,          -- Total cVault shares outstanding
  depositor_count INTEGER NOT NULL,       -- Number of unique depositors
  core_alloc NUMERIC DEFAULT 0.70,        -- Core tier allocation (0-1)
  mid_alloc NUMERIC DEFAULT 0.20,         -- Mid-Risk tier allocation (0-1)
  degen_alloc NUMERIC DEFAULT 0.10,       -- Degen tier allocation (0-1)
  idle_cash NUMERIC DEFAULT 0
);

-- Index for latest snapshot queries
CREATE INDEX idx_vault_snapshots_timestamp ON vault_snapshots(timestamp DESC);
```

user_positions

```sql
CREATE TABLE user_positions (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  wallet_address TEXT NOT NULL UNIQUE,
  shares NUMERIC NOT NULL,                -- cVault shares held
  deposited_value NUMERIC NOT NULL,       -- Total USD deposited (lifetime)
  last_updated TIMESTAMPTZ DEFAULT now()
);

CREATE INDEX idx_user_positions_wallet ON user_positions(wallet_address);
```

deposits

```sql
CREATE TABLE deposits (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  tx_hash TEXT,
  wallet_address TEXT NOT NULL,
  asset TEXT NOT NULL,                     -- "ETH" or "USDC"
  amount NUMERIC NOT NULL,                -- Amount in USD
  shares_received NUMERIC NOT NULL,       -- cVault shares minted
  timestamp TIMESTAMPTZ DEFAULT now()
);

CREATE INDEX idx_deposits_wallet ON deposits(wallet_address);
```

withdrawals

```sql
CREATE TABLE withdrawals (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  tx_hash TEXT,
  wallet_address TEXT NOT NULL,
  shares_burned NUMERIC NOT NULL,
  assets_received NUMERIC NOT NULL,       -- USD value returned
  fee_charged NUMERIC DEFAULT 0,
  timestamp TIMESTAMPTZ DEFAULT now()
);

CREATE INDEX idx_withdrawals_wallet ON withdrawals(wallet_address);
```

trades

```sql
CREATE TABLE trades (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  tx_hash TEXT,
  timestamp TIMESTAMPTZ DEFAULT now(),
  action_type TEXT NOT NULL CHECK (action_type IN ('swap', 'add_lp', 'remove_lp', 'stake',
'unstake')),
  asset_in TEXT NOT NULL,
  asset_out TEXT NOT NULL,
  amount_in NUMERIC NOT NULL,
  amount_out NUMERIC,
  protocol TEXT NOT NULL,
  tier TEXT NOT NULL CHECK (tier IN ('core', 'mid', 'degen')),
  reasoning TEXT,
  reasoning_hash TEXT,
  confidence NUMERIC,
  pnl NUMERIC DEFAULT 0,
  status TEXT DEFAULT 'executed'
);

CREATE INDEX idx_trades_timestamp ON trades(timestamp DESC);
CREATE INDEX idx_trades_tier ON trades(tier);
```

ai_reasoning_logs

```sql
CREATE TABLE ai_reasoning_logs (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  timestamp TIMESTAMPTZ DEFAULT now(),
  cycle_id INTEGER,
  market_summary TEXT NOT NULL,
  sentiment_data JSONB,                   -- { "ETH": 0.72, "USDC": 0.1, ... }
  risk_assessment JSONB,                  -- { "volatility": 0.12, ... }
  decision TEXT NOT NULL CHECK (decision IN ('hold', 'trade', 'rebalance')),
  confidence NUMERIC NOT NULL,
  trades_proposed JSONB,
  trades_executed JSONB
);

CREATE INDEX idx_ai_logs_timestamp ON ai_reasoning_logs(timestamp DESC);
```

governance_proposals

```sql
CREATE TABLE governance_proposals (
  id TEXT PRIMARY KEY,                    -- e.g., "PROP-001"
  proposal_id INTEGER,
  proposer TEXT NOT NULL,
  title TEXT NOT NULL,
  description TEXT,
  votes_for NUMERIC DEFAULT 0,
  votes_against NUMERIC DEFAULT 0,
  status TEXT NOT NULL CHECK (status IN ('active', 'passed', 'rejected', 'queued')),
  created_at TIMESTAMPTZ NOT NULL,
  voting_ends_at TIMESTAMPTZ NOT NULL
);
```

staking_positions

```sql
CREATE TABLE staking_positions (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  wallet_address TEXT NOT NULL,
  amount NUMERIC NOT NULL,                -- CORTEX tokens staked
  lock_duration INTEGER NOT NULL,         -- Days (0, 30, 90, 180)
  multiplier NUMERIC NOT NULL,            -- 1x, 1.5x, 2x, 2.5x
  rewards_claimed NUMERIC DEFAULT 0,
  staked_at TIMESTAMPTZ DEFAULT now(),
  unlocks_at TIMESTAMPTZ
);

CREATE INDEX idx_staking_wallet ON staking_positions(wallet_address);
```

RLS Policies (apply to ALL tables)

```sql
-- Allow public reads on all tables
ALTER TABLE vault_snapshots ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Public read" ON vault_snapshots FOR SELECT USING (true);

-- Repeat for every table:
-- user_positions, deposits, withdrawals, trades,
-- ai_reasoning_logs, governance_proposals, staking_positions
```

---

7. Smart Contracts (Must Deploy)

Five contracts need to be written and deployed to Base mainnet:

7.1 CortexVault.sol (ERC-4626)

The core vault contract. Users deposit ETH/USDC, receive cVault shares.

Key Functions:

```
// Inherits from ERC4626 (OpenZeppelin)
function deposit(uint256 assets, address receiver) external returns (uint256 shares);
```

```
function redeem(uint256 shares, address receiver, address owner) external returns (uint256 assets);
function totalAssets() public view returns (uint256);

// AI Agent execution (restricted to AI_ROLE)
function executeStrategy(bytes calldata data) external onlyRole(AI_ROLE);

// Emergency brake (governance)
function pause() external;
function unpause() external;
```

Requirements:

- Accept ETH (wrap to WETH) and USDC
- Mint cVault shares proportional to deposit
- Support withdrawal with 0.5% fee
- `executeStrategy()` callable only by AI agent address
- Pausable via governance emergency brake
- Emit events for all deposits/withdrawals (indexer listens to these)

Recommended: Use OpenZeppelin ERC4626 + AccessControl + Pausable

7.2 CortexToken.sol (ERC-20)

Governance token with fixed supply.

```
// Fixed supply: 1,000,000,000 CORTEX
// No mint function (fixed supply)
// Standard ERC-20 with ERC20Votes extension for governance
```

7.3 Staking.sol

Lock CORTEX tokens to earn protocol fee share.

```
function stake(uint256 amount, uint256 lockDuration) external;
function unstake() external;  // Only after lock expires
function claimRewards() external;
function getPosition(address user) external view returns (StakePosition memory);
```

Lock tiers:

Duration	Multiplier
No lock	1.0x
30 days	1.5x
90 days	2.0x
180 days	2.5x

7.4 Governor.sol

On-chain governance for parameter changes.

```
// Use OpenZeppelin Governor + GovernorVotes + GovernorTimelockControl
// Quorum: 4% of total supply
// Voting period: 3 days
```

```
// Timelock: 24 hours
// Emergency brake: 66% supermajority can pause vault
```

7.5 Treasury.sol

Collects and distributes protocol fees.

```
function distributeFees() external;  // 50% to stakers, 50% to treasury
function withdraw(address token, uint256 amount) external onlyGovernance;
```

Deployment Steps

1. Write contracts using Foundry or Hardhat
2. Deploy to Base Sepolia testnet first
3. Verify on Basescan
4. Audit (recommended: Trail of Bits, OpenZeppelin, or Code4rena)
5. Deploy to Base mainnet
6. Update `src/lib/constants.ts` with real addresses
7. Update `src/lib/token-gate.ts` to use real CORTEX_TOKEN address

After Deployment — Frontend Wiring

Add Wagmi hooks in `src/app/vault/page.tsx`:

```
import { useWriteContract, useWaitForTransactionReceipt } from "wagmi";
import { parseEther, parseUnits } from "viem";
import { CONTRACTS } from "@/lib/constants";
import { vaultAbi } from "@/lib/abis/vault";

// Deposit ETH
const { writeContract: deposit } = useWriteContract();
deposit({
  address: CONTRACTS.CVAULT,
  abi: vaultAbi,
  functionName: "deposit",
  args: [parseEther(amount), address],
  value: parseEther(amount), // for ETH deposits
});

// Deposit USDC (requires approval first)
// 1. Approve USDC spend
// 2. Call vault.deposit()

// Withdraw
const { writeContract: redeem } = useWriteContract();
redeem({
  address: CONTRACTS.CVAULT,
  abi: vaultAbi,
  functionName: "redeem",
  args: [shares, address, address],
});
```

---

8. AI Agent System

The AI agent is the core of CORTEX. It does NOT exist yet. Here's the design:

8.1 What the AI Agent Does

Every 10 minutes:

1. **Read on-chain state** — vault TVL, current allocations, token prices
2. **Analyze market data** — price feeds, volume, liquidity depth, sentiment (Farcaster, Twitter)
3. **Generate trade proposals** — rebalance tiers, enter/exit positions
4. **Validate against guardrails** — max slippage, tier bounds, rate limits
5. **Execute trades** — call vault's `executeStrategy()` via the AI agent wallet
6. **Log reasoning** — write to `ai_reasoning_logs` table in Supabase

8.2 Recommended Architecture

```
┌──────────────────────────────────────────┐
│           AI AGENT (Python/Node.js)       │
│                                          │
│  1. Data Collection Layer                │
│     - Viem/ethers for on-chain reads     │
│     - CoinGecko/DexScreener for prices   │
│     - Farcaster API for sentiment        │
│     - Supabase for historical data       │
│                                          │
│  2. Analysis Layer                       │
│     - LLM (Claude API) for reasoning     │
│     - Custom ML for sentiment scoring    │
│     - Risk model (volatility, drawdown)  │
│                                          │
│  3. Decision Layer                       │
│     - Trade proposal generation          │
│     - Guardrail validation               │
│     - Confidence scoring                 │
│                                          │
│  4. Execution Layer                      │
│     - Viem for transaction signing       │
│     - Multi-sig or timelock for safety   │
│     - Supabase logging                   │
│                                          │
│  5. Logging Layer                        │
│     - Write to ai_reasoning_logs         │
│     - Write to trades table              │
│     - Update vault_snapshots             │
└──────────────────────────────────────────┘
```

8.3 Agent Decision Flow

```
Input Data → Sentiment Analysis → Risk Assessment → Trade Decision
                                                        │
                                          ┌─────────────┼─────────────┐
                                          │             │             │
                                        HOLD          TRADE        REBALANCE
                                          │             │             │
                                     Log only      Execute swap    Move between
                                                   Add/Remove LP   tiers
                                                   Stake/Unstake
```

8.4 Guardrails (Enforced by Smart Contract)

- Core allocation must stay within 50-90%
- Mid-Risk allocation must stay within 5-35%
- Degen allocation must stay within 0-15%
- Max slippage per trade: 1.5%
- Max 20 trades per hour
- Cannot trade paused tokens
- Cannot exceed tier bounds without governance vote

8.5 Running the Agent

Options:

- **Railway/Render** — run as a long-running service with cron
- **AWS Lambda + EventBridge** — serverless, runs every 10 minutes
- **VPS (DigitalOcean/Hetzner)** — most control, cheapest for always-on
- **Vercel Cron** — limited to 60s execution, may not be enough for complex analysis

The agent needs its own wallet (private key) with:

- ETH for gas on Base
- Whitelisted as `AI_ROLE` in the vault contract

---

9. Cron Jobs (All Placeholder)

Five cron routes exist but contain only TODO comments:

Route	Purpose	Writes To	Priority
`/api/cron/snapshot`	Read vault contract → write TVL/share price	`vault_snapshots`	HIGH
`/api/cron/prices`	Fetch token prices from DEX/oracle	`token_prices`	HIGH
`/api/cron/fees`	Calculate + distribute protocol fees	`fee_distributions`	MEDIUM
`/api/cron/governance`	Sync on-chain vote tallies	`governance_proposals`	MEDIUM
`/api/cron/analytics`	Aggregate performance metrics	analytics tables	LOW

All are authenticated with `Authorization: Bearer <CRON_SECRET>`.

Implementation order: snapshot → prices → fees → governance → analytics

These should run on Vercel Cron (configured in `vercel.json`):

```
{
  "crons": [
    { "path": "/api/cron/snapshot", "schedule": "*/5 * * * *" },
    { "path": "/api/cron/prices", "schedule": "*/2 * * * *" },
    { "path": "/api/cron/fees", "schedule": "0 0 * * 0" },
    { "path": "/api/cron/governance", "schedule": "*/10 * * * *" },
    { "path": "/api/cron/analytics", "schedule": "0 * * * *" }
  ]
}
```

---

10. Frontend → API Mapping

Every page fetches from specific API routes:

Page	Route File	API Endpoints Called
Landing (/)	src/app/page.tsx	GET /api/v1/vault/stats
Dashboard (/dashboard)	src/app/dashboard/page.tsx	GET /api/v1/vault/stats + /portfolio + /performance + /vault/user/[addr]
Vault (/vault)	src/app/vault/page.tsx	GET /api/v1/vault/stats + /performance + /vault/user/[addr]
Trades (/trades)	src/app/trades/page.tsx	GET /api/v1/trades?page=&pageSize=&type=&tier=
AI (/ai)	src/app/ai/page.tsx	GET /api/v1/ai/insights + /ai/reasoning-feed (requires x-wallet-address header)
Governance (/governance)	src/app/governance/page.tsx	GET /api/v1/governance/proposals
Stake (/stake)	src/app/stake/page.tsx	GET /api/v1/staking/info/[addr]
Strategy (/strategy)	src/app/strategy/page.tsx	GET /api/v1/portfolio

API Route → Supabase Table Mapping

API Route	Supabase Table(s)	Fallback
/api/v1/vault/stats	vault_snapshots	mockVaultStats
/api/v1/vault/user/[addr]	user_positions + vault_snapshots + deposits + withdrawals	mockUserPosition
/api/v1/portfolio	vault_snapshots	mockPortfolio
/api/v1/trades	trades	mockTrades
/api/v1/trades/[id]	trades	mockTrades
/api/v1/ai/insights	ai_reasoning_logs	mockAiInsights
/api/v1/ai/reasoning-feed	ai_reasoning_logs	mockReasoningFeed
/api/v1/governance/proposals	governance_proposals	mockGovernance
/api/v1/staking/info/[addr]	staking_positions	mockStakingInfo
/api/v1/performance	vault_snapshots	mockPerformance

---

11. Environment Variables

Required in Vercel + `.env.local`

```
# Supabase
NEXT_PUBLIC_SUPABASE_URL=https://qyqjydtmhumtpxfwlfbx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=<anon key>
SUPABASE_SERVICE_ROLE_KEY=<service role key>

# Base RPC (for on-chain reads)
BASE_RPC_URL=https://mainnet.base.org
```

```
# Cron job authentication
CRON_SECRET=<generate a random string>

# Wallet Connect (for frontend wallet connection)
NEXT_PUBLIC_WALLETCONNECT_PROJECT_ID=<from cloud.walletconnect.com>

# AI Agent (when built)
AI_AGENT_PRIVATE_KEY=<private key for agent wallet>
ANTHROPIC_API_KEY=<for Claude API calls in AI agent>
```

Currently Set in Vercel

- `NEXT_PUBLIC_SUPABASE_URL` ✅
- `NEXT_PUBLIC_SUPABASE_ANON_KEY` ✅
- `SUPABASE_SERVICE_ROLE_KEY` ✅

Missing from Vercel

- `CRON_SECRET` ❌
- `AI_AGENT_PRIVATE_KEY` ❌
- `ANTHROPIC_API_KEY` ❌

---

12. Implementation Roadmap

Phase 1: Fix Data Display (1-2 days)

Goal: Make the website show real data from Supabase instead of mock fallbacks.

1. **Verify Supabase tables** — run the SQL from Section 6 to create/recreate tables with correct column names
2. **Configure RLS** — add public read policies on all tables
3. **Add error logging** — add `console.error()` to all API route catch blocks
4. **Seed real data** — insert realistic sample data into all tables
5. **Test each API route** — `curl https://www.cortexprotocol.net/api/v1/vault/stats` and verify non-mock response
6. **Deploy and verify** — push to GitHub → Vercel auto-deploys

Phase 2: Smart Contracts (1-3 weeks)

Goal: Deploy vault, token, staking, governance, and treasury contracts.

1. **Set up Foundry project** — `forge init cortex-contracts`
2. **Write CortexVault.sol** — ERC-4626 with AI execution role
3. **Write CortexToken.sol** — ERC-20 with votes extension, 1B fixed supply
4. **Write Staking.sol** — lock periods, multipliers, reward distribution
5. **Write Governor.sol** — OpenZeppelin Governor stack
6. **Write Treasury.sol** — fee collection + distribution
7. **Write tests** — full coverage for all contracts
8. **Deploy to Base Sepolia** — test everything
9. **Security audit** — at least one professional audit
10. **Deploy to Base mainnet** — update `constants.ts` with real addresses

Phase 3: Wire Deposits/Withdrawals (2-3 days, after contracts)

Goal: Make the Vault page functional for real deposits and withdrawals.

1. **Create ABI files** — `src/lib/abis/vault.ts`, `src/lib/abis/token.ts`, etc.
2. **Add Wagmi hooks** — `useWriteContract` for deposit, redeem, approve
3. **Wire Vault page** — connect deposit/withdraw buttons to contract calls

4. **Add transaction tracking** — show pending/confirmed state, link to Basescan
5. **Wire Staking page** — connect stake/unstake/claim buttons
6. **Wire Governance page** — connect vote buttons to Governor contract
7. **Add event indexer** — listen for Deposit/Withdraw events, write to Supabase

Phase 4: Build AI Agent (2-4 weeks)

Goal: Create the autonomous AI agent that manages the vault.

1. **Choose runtime** — Python (recommended) or Node.js
2. **Build data collection** — on-chain reads, price feeds, sentiment
3. **Build analysis engine** — integrate Claude API for reasoning
4. **Build execution layer** — sign and send transactions via agent wallet
5. **Build logging** — write all decisions to `ai_reasoning_logs`
6. **Build guardrail validation** — enforce tier bounds, slippage limits
7. **Test on testnet** — run agent against Base Sepolia vault
8. **Deploy to production** — Railway/Render/VPS

Phase 5: Implement Cron Jobs (1-2 days)

Goal: Automate data pipelines.

1. **Implement `/api/cron/snapshot`** — read vault contract, write to `vault_snapshots`
2. **Implement `/api/cron/prices`** — fetch from DEX, write to `token_prices`
3. **Implement `/api/cron/fees`** — calculate fees, write distributions
4. **Implement `/api/cron/governance`** — sync on-chain vote data
5. **Implement `/api/cron/analytics`** — aggregate performance metrics
6. **Configure `vercel.json`** — set up cron schedules

Phase 6: Production Hardening

1. **Add rate limiting** to API routes
2. **Add input validation** with zod
3. **Add monitoring** — Vercel analytics, Supabase alerts
4. **Add Supabase Realtime** — already partially implemented via `RealtimeProvider`
5. **Security audit** of both contracts and frontend
6. **Bug bounty program**

---

13. Deployment

Current Setup

- **GitHub:** Push to `main` branch at `CortexProtocolBase/Cortex-Protocol`
- **Vercel:** Auto-deploys on push (connected to GitHub)
- **Manual deploy:** `npx vercel --prod` from project root
- **Domain:** `cortexprotocol.net` aliased in Vercel

Git Identity

All commits must use:

```
git -c user.name="CortexProtocolBase" -c user.email="Cortex-dev@outlook.com" commit -m "message"
```

Key File Locations

```
src/
├── app/
│   ├── page.tsx                    # Landing page
```

```
│   ├── dashboard/page.tsx          # Dashboard
│   ├── vault/page.tsx              # Deposit/Withdraw
│   ├── trades/page.tsx             # Trade history
│   ├── ai/page.tsx                 # AI insights
│   ├── governance/page.tsx         # Governance
│   ├── stake/page.tsx              # Staking
│   ├── strategy/page.tsx           # Portfolio/Strategy
│   └── api/
│       ├── v1/
│       │   ├── vault/stats/route.ts
│       │   ├── vault/user/[address]/route.ts
│       │   ├── portfolio/route.ts
│       │   ├── trades/route.ts
│       │   ├── trades/[id]/route.ts
│       │   ├── ai/insights/route.ts
│       │   ├── ai/reasoning-feed/route.ts
│       │   ├── governance/proposals/route.ts
│       │   ├── staking/info/[address]/route.ts
│       │   └── performance/route.ts
│       └── cron/
│           ├── snapshot/route.ts    # TODO
│           ├── prices/route.ts      # TODO
│           ├── fees/route.ts        # TODO
│           ├── governance/route.ts  # TODO
│           └── analytics/route.ts   # TODO
├── components/
│   ├── Navbar.tsx
│   ├── Toast.tsx                   # Toast notification system
│   └── RealtimeProvider.tsx        # Supabase Realtime
├── lib/
│   ├── constants.ts                # Contract addresses (ALL ZERO), fees, tiers
│   ├── mock-data.ts                # Fallback mock data for all endpoints
│   ├── supabase.ts                 # Supabase client (public + admin)
│   ├── token-gate.ts               # Token balance check for gated endpoints
│   ├── types.ts                    # All TypeScript types and interfaces
│   └── wagmi.ts                    # Wagmi config (Phantom + Coinbase)
```

Quick Start for New Developer

```
# Clone
git clone https://github.com/CortexProtocolBase/Cortex-Protocol.git
cd Cortex-Protocol

# Install
npm install

# Copy env (get values from Vercel dashboard)
cp .env.example .env.local

# Run dev server
npm run dev

# Build
npm run build
```

```
# Deploy
npx vercel --prod
```

First Priority Actions

1. Open Supabase dashboard → SQL Editor → run all CREATE TABLE statements from Section 6
2. Add RLS policies (SELECT for public)
3. Insert seed data into all tables
4. Add `console.error` logging to all API route catch blocks
5. Test: `curl http://localhost:3000/api/v1/vault/stats` — should return real data, not mock
6. Deploy

This document was generated for the CORTEX Protocol developer handoff. For questions, refer to the GitHub repository or Vercel project dashboard.